# SOFTWARE ENGINEERING

# Assignment 3

# CSA10 – SOFTWARE ENGINEERING

# ASSIGNMENT 3

CO3: Develop full-stack applications using version control systems like git/GitHub and containerization tools like Docker, Kubernetes.

**Assignment Title:** Design, Develop and Validate a Containerized Full Stack Smart Campus Facility Booking and Maintenance Platform

**Nature of Assignment:** Individual

**Duration:** One week

**Total Marks:** 50

**Submission Date:** _____

## Scenario — Integrated Campus Operations Development and Container Deployment

A university requires a working full-stack campus operations release that supports facility booking or sensor-alert intake, explainable maintenance triage, technician assignment, inspection, approval and verified completion. Separate frontend, backend and database changes create integration problems, while inconsistent developer environments make deployment difficult to reproduce. Develop an integrated application with persistent records, server-enforced permissions and valid booking and ticket transitions. Establish a traceable Git workflow and a reproducible container deployment. Adapt the implementation for duplicate requests, overlapping reservations, conflicting updates and two backend instances, validating that retries and restarts preserve booking state, ticket state and audit history.

The university proposes an AI-assisted Smart Campus Facility and Maintenance Platform. Its intended modules are an operations dashboard, a facility and room catalogue, booking management, maintenance ticket lifecycle, utility and sensor monitoring, equipment loan requests, an approval workflow and a DevOps integration hub. The system should connect a booking problem or sensor alert to a tracked maintenance ticket, an authorized inspection, a justified approval, deployment verification and closure.

Implementation conditions for the representative release are stated below.

| Observation | Scenario planning condition |
|---|---|
| Operational scope | Facility booking, sensor-alert intake, maintenance tickets, approvals, equipment loans and DevOps links |
| Users | Administrator, facility manager, maintenance technician, department approver, faculty member and student |
| Reference data | Synthetic facility, booking and ticket records and controlled sensor events |

| Observation | Scenario planning condition |
|---|---|
| Integration constraint | External issue tracker or calendar service may be unavailable; preserve local records and retry safely |
| Validation need | Verify booking overlap, priority, status, SLA, authorization, persistence and audit behaviour |

Principal users are campus administrators, facility managers, maintenance technicians, department approvers, faculty members and students. They need different permissions. Facility managers require useful triage suggestions; approvers require auditable decisions; engineers require traceable links to development issues and releases. Automation must preserve human responsibility for prioritization, approval and operational changes.

The application must distinguish users, facilities or assets, sensor alerts, bookings, maintenance tickets, SLA policies, approvals, integration links and audit events. Duplicate sensor messages, repeated booking submissions or repeated integration requests must not silently create inconsistent records. Two users attempting to reserve the same facility slot, and concurrent technician updates, must be handled explicitly.

The development team also faces integration problems. Changes to the user interface, backend API and database may be developed separately, and an application that works on one contributor’s computer may fail on another.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, utilisation figures, triage scores and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live monitoring or secure authorization. Verify claims through repeatable checks. Distinguish deterministic keyword rules from trained machine learning and label simulations and proposed extensions accurately.

## Design Challenge:

The university requires a working first release supporting facility booking or sensor-alert intake, assisted triage, assignment, SLA visibility, resolution and closure. Demonstrate one linked approval or DevOps operation. Integrate the interface, API and persistent database and reproduce execution in a clean environment.

Design, develop and validate a containerized full-stack application that addresses these conditions.

Explore alternative technology choices and select a suitable frontend framework, backend framework, database, version-control platform and container deployment approach. Justify your selections against integration needs, maintainability, reproducibility, available resources and cost.

Create a working application with traceable version history and reproducible deployment. Construct scenarios exposing duplicate requests, overlapping bookings, conflicting status changes, unauthorized actions and environment failures. Revise the implementation using observed evidence.

### Q1 – Define the Application Scope and Select the Technology Stack 5 Marks

Investigate the campus facility booking and maintenance process and define a feasible first release.

- Identify principal users, workflows and implementation constraints.
- Define at least six user stories with measurable acceptance criteria.
- Establish scope exclusions appropriate to the one-week assignment.
- Compare at least two feasible technology-stack combinations.
- Select and justify frontend, backend, database and supporting tools.
- Define measurable criteria for functional correctness and deployment reproducibility.

Use a version-control system and a containerization approach appropriate to the selected solution. Explain how they support the capabilities described in CO3.

#### Evidence Expected

User stories, acceptance criteria, scope boundaries, technology comparison, justified selections and success criteria.

### Q2 – Create the Integrated Application Design 7 Marks

Develop a design connecting the interface, backend API and persistent data. Include:

- A component diagram showing principal application components.
- A data model distinguishing users, facilities, sensor alerts, bookings, maintenance tickets, SLA policies, approvals and integration links.
- API contracts for principal operations.
- A booking state model (requested, approved, confirmed, in use, completed, cancelled) and a ticket state model covering intake, assignment, inspection, pending work, resolution and closure.
- Rules for valid transitions, duplicate requests, overlapping reservations, competing updates and approval-gated actions.
- An explanation of how the system prevents double booking and duplicate ticket creation for the same event and preserves audit history.

Define how validation responsibilities are distributed between the frontend, backend and database.

#### Evidence Expected

Component diagram, data model, API contracts, state models and justified consistency rules.

### Q3 – Develop a Working Full-Stack Application 12 Marks

Implement a complete workflow that supports:

- Viewing the facility catalogue, booking calendar, maintenance queue, SLA indicators and dashboard.

- Submitting a booking, or generating a maintenance ticket from a controlled sensor threshold event.
- Assigning a ticket and producing explainable category and priority suggestions.
- Recording inspection notes, resolution and authorized closure.
- Submitting and approving or rejecting a restricted-facility booking, equipment loan or change using role restrictions.
- Viewing a linked developer issue or delivery status with disclosed integration mode.

Your implementation must demonstrate:

- Frontend interaction with the backend API.
- Persistent storage of application data.
- Appropriate user roles and access restrictions.
- Backend validation of critical operations, including overlap checks.
- Clear handling of rejected operations and integration errors.
- Consistent booking, ticket, approval and audit state after updates and retries.

Demonstrate one complete journey through the actual application, from a booking or sensor alert to a closed ticket, including the selected linked approval or DevOps step.

### Evidence Expected

Working source code, execution evidence, persistent records and a demonstrated end-to-end workflow.

## Q4 – Establish and Demonstrate a Version-Control Workflow 6 Marks

Create a repository and design a workflow suitable for contributions.

- Organize the repository and exclude generated files and sensitive configuration.
- Create at least two feature branches representing distinct changes.
- Make meaningful commits with clear messages.
- Submit at least one change for review using the selected repository platform.
- Demonstrate a merge and resolve one deliberately introduced conflict.
- Identify a stable release using a tag or equivalent version identifier.

For this individual assignment, simulate separate contributions transparently. Do not present self-review as review by another person.

Explain how the workflow reduces integration risk and supports recovery from an unsuitable change.

### Evidence Expected

Repository structure, commit history, branches, review record, conflict-resolution evidence and stable release identifier.

## Q5 – Create and Validate a Reproducible Container Deployment 8 Marks

Containerize the application using your selected tools.

- Create build definitions for application components.
- Define service networking, configuration and persistent storage.
- Manage sensitive values separately from committed source code.
- Provide startup, shutdown and clean-environment deployment instructions.
- Deploy the application from the identified release.
- Verify that required data survives an application-container restart.
- Demonstrate that a fresh deployment reproduces the principal booking and maintenance workflow.

Explain your deployment choice. A small first release may use a multi-container arrangement; orchestration should be introduced only where its operational benefits are justified.

### Evidence Expected

Container build and deployment files, configuration guidance, deployment commands, execution evidence and persistence verification.

## Q6 – Adapt the Application for Concurrent Access and Deployment Changes 8 Marks

### Changed Requirement

The university expands to six campuses. The planning peak grows from 60 to 300 users, and the backend must operate with two instances sharing the database. During operation:

- Two users attempt to reserve the same laboratory for overlapping time slots, and two technicians submit incompatible updates to the same ticket.
- A sensor event or developer issue creation request is retried after a timeout.
- One backend instance restarts while the other continues serving users.

Create revised implementation and deployment that preserve booking and ticket consistency, prevent duplicate effects and retain audit history.

| Component | Required Work | Marks |
|---|---|---|
| A. Diagnose the impact | Identify affected assumptions, application state, booking and ticket update logic and deployment configuration. | 2 |
| B. Create the adaptation | Revise critical overlap, update and deduplication logic and deployment configuration for two backend instances and safe retries. | 3 |
| C. Validate and defend | Demonstrate overlapping reservations, conflicting updates, duplicate retries and instance restart. Interpret results, limitations and trade-offs. | 3 |

Validate increased load through a feasible bounded experiment and report the actual workload. Do not claim support for 300 simultaneous users without corresponding evidence.

### Evidence Expected

Impact analysis, revised code and deployment files, concurrency and restart evidence, and a justified explanation of the resulting behaviour.

### Q7 – Individual Technical Reflection 4 Marks

Explain:

- Which integration decision had the greatest effect on your application.
- Which defect or failure led to a meaningful implementation change.
- How version control and containerization improved reproducibility.
- Which limitation should be addressed before wider deployment.

### Evidence Expected

A concise reflection on your application, development workflow and deployment limitations. Refer to your own implementation and evidence.

## MARK DISTRIBUTION

| Question | Assessed Area | Marks |
|---|---|---|
| Q1 | Scope definition and justified technology selection | 5 |
| Q2 | Integrated application and data design | 7 |
| Q3 | Working full-stack implementation | 12 |
| Q4 | Version-control workflow and integration evidence | 6 |
| Q5 | Reproducible container deployment | 8 |
| Q6 | Day-4 Design Challenge – adaptation for concurrency and deployment changes | 8 |
| Q7 | Individual technical reflection | 4 |
| | **TOTAL** | **50** |

### Assessment Verification

Students may be asked to trace a booking or ticket from interface to database, explain a consistency or authorization rule, demonstrate a merge, reproduce deployment or diagnose a deployment failure.

### Overall Assessment Expectation

Investigate > Design > Develop > Integrate > Containerize > Validate > Adapt > Defend

The final submission must demonstrate an original working solution aligned with CO3, BTL 5 – Evaluate and BTL 6 – Create.